NP완전 문제

여러 선택을 바꿔 보며 조건에 맞는 답을 구합니다.

답을 찾기

답을 찾기

여러 선택을 바꿔 보며 조건에 맞는 답을 구합니다.

답을 확인하기

이미 준 선택이 맞는지 계산만 합니다. 앞의 세 문제에서 이 차이를 봅니다.

9-1 주어진 선택의 합 확인

정수 배열 a와 같은 길이의 선택 표시 selected가 주어진다. selected[i]=1이면 a[i]를 골랐고, 0이면 고르지 않았다. 이미 고른 원소들의 합이 target과 같은지 검사한다.

고른 원소의 합이 target이면 1, 다르면 0을 반환하시오.

9-1 입력과 반환값

Python: verify_subset_sum(a, selected, target)

a: 길이 0~20인 배열. 각 원소는 0~100의 정수
selected: a와 길이가 같은 0/1 배열
target: 검사할 목표 합. 0~2000의 정수

반환값

검사 결과인 정수 1 또는 0을 반환한다. 원소의 합 자체를 반환하지 않는다.

선택·판정 조건

선택 표시를 바꾸거나 다른 부분집합을 새로 찾지 않는다.
선택한 원소가 없을 때의 합은 0이다.
값이 0인 원소를 선택해도 합은 변하지 않는다.

9-1 고른 합이 목표와 같음 · 목표만 다름

고른 합이 목표와 같음

a=[1,2,3], selected=[1,0,1], target=4
반환값: 1
선택된 첫째와 셋째 값의 합은 1+3=4이다.

목표만 다름

같은 a와 selected, target=5
반환값: 0
선택 합은 여전히 4이므로 목표 5와 다르다.

9-1 선택을 새로 찾지 않음 · 빈 선택과 목표 0

선택을 새로 찾지 않음

a=[1,2,3], selected=[0,0,0], target=3
반환값: 0
값 3이 배열에 있어도 선택되지 않았다. 현재 합은 0이다.

빈 선택과 목표 0

a=[], selected=[], target=0
반환값: 1
빈 합 0이 목표와 같다.

9-1 0을 선택함

0을 선택함

a=[0], selected=[1], target=0
반환값: 1
선택한 값 0의 합도 0이다.

9-1 예제의 계산 과정

배열의 값	선택 표시	이번에 더함	누적 합
1	1	1	1
2	0	0	1
3	1	3	4

함수가 새로 고를 수를 찾는 작업은 없습니다. 주어진 선택만 검사합니다.

9-1 표시가 1인 원소만 더한다

선택 표시가 이미 주어졌으므로 가능한 선택들을 탐색할 필요가 없다. 배열을 한 번 읽으며 표시가 1인 값만 합에 더한다. 마지막에 이 합을 target과 비교한다. 7장의 부분집합 합 찾기와 달리, 여기서는 제시된 답 하나를 검사한다.

9-1 코드에서 사용할 값

코드	뜻
selected[i]	i번 원소를 골랐는지 표시한 0 또는 1
total += a[i]	선택한 원소의 값 더하기
total == target	선택된 값의 합이 목표와 같은지 비교

맞는 답을 제시받으면 확인만 하면 됩니다.

9-1 Python

```
def verify_subset_sum(a, selected, target):
    total = 0
    for i in range(len(a)):
        if selected[i]:
            total += a[i]
    return int(total == target)
```

시간 O(n), 추가 공간 O(1)

9-1 C 함수의 인자

함수 원형

int verify_subset_sum(const int a[], const int selected[], int n, int target)

배열과 작업 공간

a는 값 배열, selected는 선택 표시이다. 두 배열의 길이는 n이다.

9-1 C

```
int verify_subset_sum(const int a[], const int selected[],
    int n, int target) {
    int total=0;
    for (int i=0; i<n; ++i) if (selected[i]) total+=a[i];
    return total==target;
}
```

시간 O(n), 추가 공간 O(1)

9-2 주어진 정점 커버 확인

그래프에서 점을 몇 개 골랐다. 각 선의 양 끝 중 적어도 한 점을 골랐으면 그 선을 덮었다고 한다. 모든 선을 덮는 점들의 선택을 정점 커버라고 한다.

selected가 모든 선을 덮고 있으면 1, 덮지 못한 선이 있으면 0을 반환하시오.

9-2 입력과 반환값

Python: verify_vertex_cover(graph, selected)

graph: 점 1~8개의 대칭인 0/1 연결 표. 대각선은 0
selected: 점 수와 같은 길이. 점 i를 골랐으면 1, 아니면 0

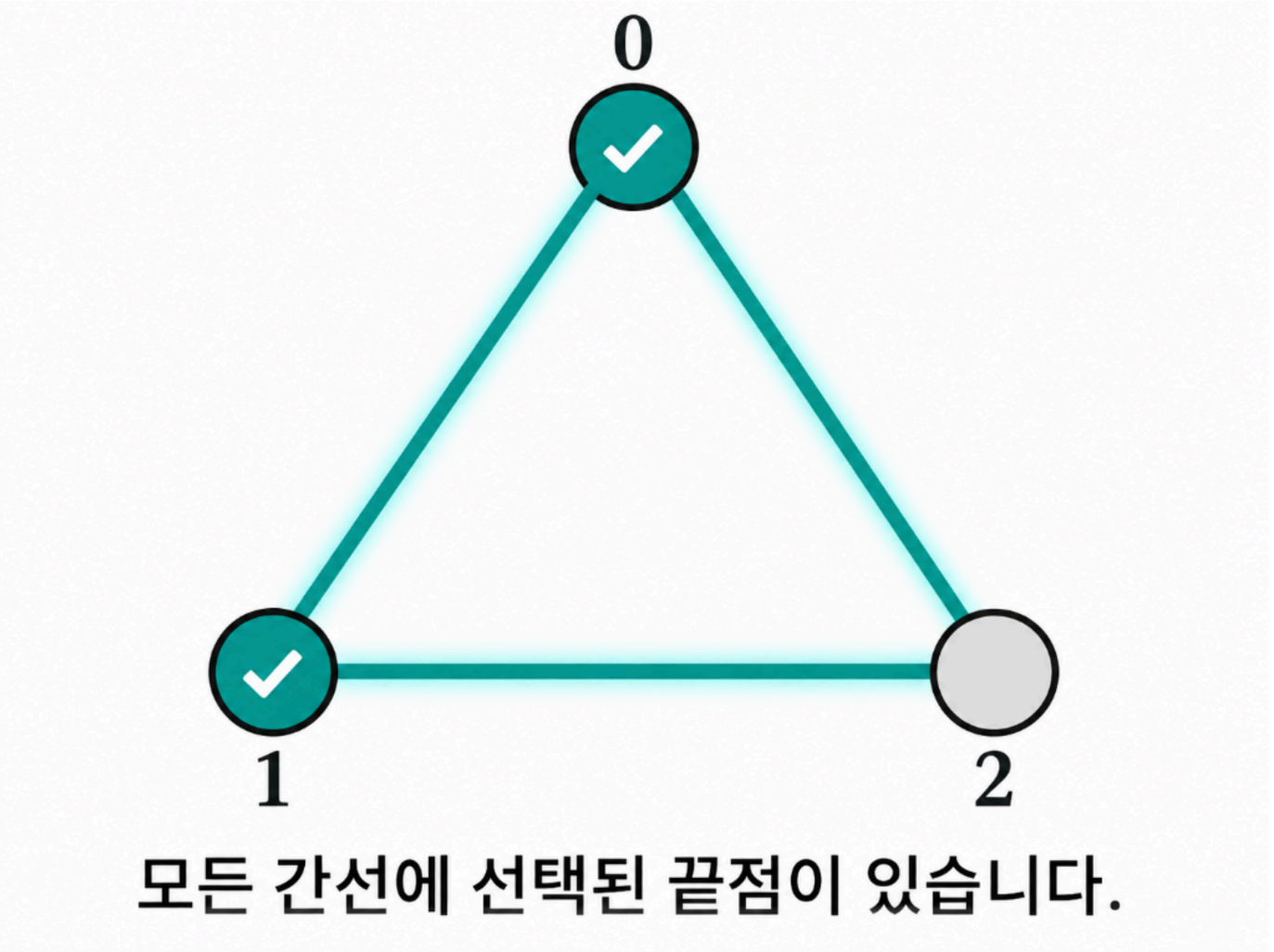
반환값

주어진 선택이 정점 커버이면 1, 아니면 0이다. 선택한 점의 수를 반환하지 않는다.

선택·판정 조건

선의 양 끝을 모두 골라도 그 선을 덮은 것이다.
점의 개수가 최소인지까지 검사하지 않는다.
선이 하나도 없으면 덮지 못한 선도 없으므로 1이다.

9-2 주어진 정점 커버 확인



삼각형에서 점 0과 1을 고르면 세 간선 모두 선택한 점에 닿는다. 점 0만 고르면 간선 1–2는 덮이지 않는다.

9-2 삼각형에서 두 점 선택 · 삼각형에서 한 점만

삼각형에서 두 점 선택

삼각형의 점0과 점1을 선택
반환값: 1
0–1,0–2,1–2 모두 선택한 끝점이 있다.

삼각형에서 한 점만

같은 삼각형에서 점0만 선택
반환값: 0
1–2 선은 양 끝을 모두 고르지 않아 덮지 못한다.

9-2 전부 선택 · 선이 없음

전부 선택

삼각형의 세 점을 모두 선택
반환값: 1
최소 개수는 아니어도 모든 선을 덮는다.

선이 없음

두 점 사이에 선이 없고 아무 점도 고르지 않음
반환값: 1
검사해야 할 선이 없어 조건을 만족한다.

9-2 예제의 계산 과정

검사할 선	끝점 중 고른 점	통과 여부
0–1	0과 1	통과
0–2	0	통과
1–2	1	통과

이처럼 모든 선에 닿도록 점을 고른 것을 정점 커버라고 부릅니다.

9-2 두 끝을 모두 고르지 않은 선 하나면 실패한다

삼각형에서 점0만 고르면 0–1과 0–2는 덮는다. 하지만 1–2는 양 끝이 모두 미선택이므로 실패한다. 코드에서는 연결된 두 점을 읽고, 두 선택 표시가 모두 0이면 바로 0을 반환한다. 끝까지 이런 선이 없으면 1이다.

9-2 코드에서 사용할 값

코드	뜻
selected[i]	점 i를 고르면 1, 아니면 0
not selected[i] and not selected[j]	두 끝점 모두 고르지 않음
j를 i+1부터 검사	무방향 선 하나를 두 번 검사하지 않음

이 문제는 주어진 선택의 검사입니다. 최소 개수의 점을 찾을 필요는 없습니다.

9-2 Python

```
def verify_vertex_cover(graph, selected):
    n = len(graph)
    for i in range(n):
        for j in range(i+1, n):
            if graph[i][j] and not selected[i] and not selected[j]:
                return 0
    return 1
```

인접 행렬 기준 시간 $O(n^2)$, 추가 공간 $O(1)$

9-2 C 함수의 인자

함수 원형

int verify_vertex_cover(const int graph[][8], int n, const int selected[])

배열과 작업 공간

graph는 연결 표, n은 정점 수, selected는 길이 n인 선택 표시이다.

9-2 C

```
int verify_vertex_cover(const int graph[][8], int n,
    const int selected[]) {
    for (int i=0; i<n; ++i)
        for (int j=i+1; j<n; ++j)
            if (graph[i][j] && !selected[i] && !selected[j])
                return 0;
    return 1;
}
```

인접 행렬 기준 시간 $O(n^2)$, 추가 공간 $O(1)$

9-3 주어진 해밀턴 순회 확인

모든 점을 정확히 한 번씩 방문한 뒤 출발점으로 돌아오는 순회를 해밀턴 순회라고 한다. 연결 표 graph와 방문 순서 path를 받아 이 순서가 올바른 해밀턴 순회인지 검사한다.

번호·중복·이동 연결을 모두 만족하면 1, 하나라도 어기면 0을 반환하시오.

9-3 입력과 반환값

Python: verify_hamiltonian(graph, path)

graph: 점 수 n이 3~8인 대칭 0/1 연결 표. 대각선은 0
path: 길이 n의 정수 배열. 잘못된 번호나 중복 번호가 들어올 수 있음

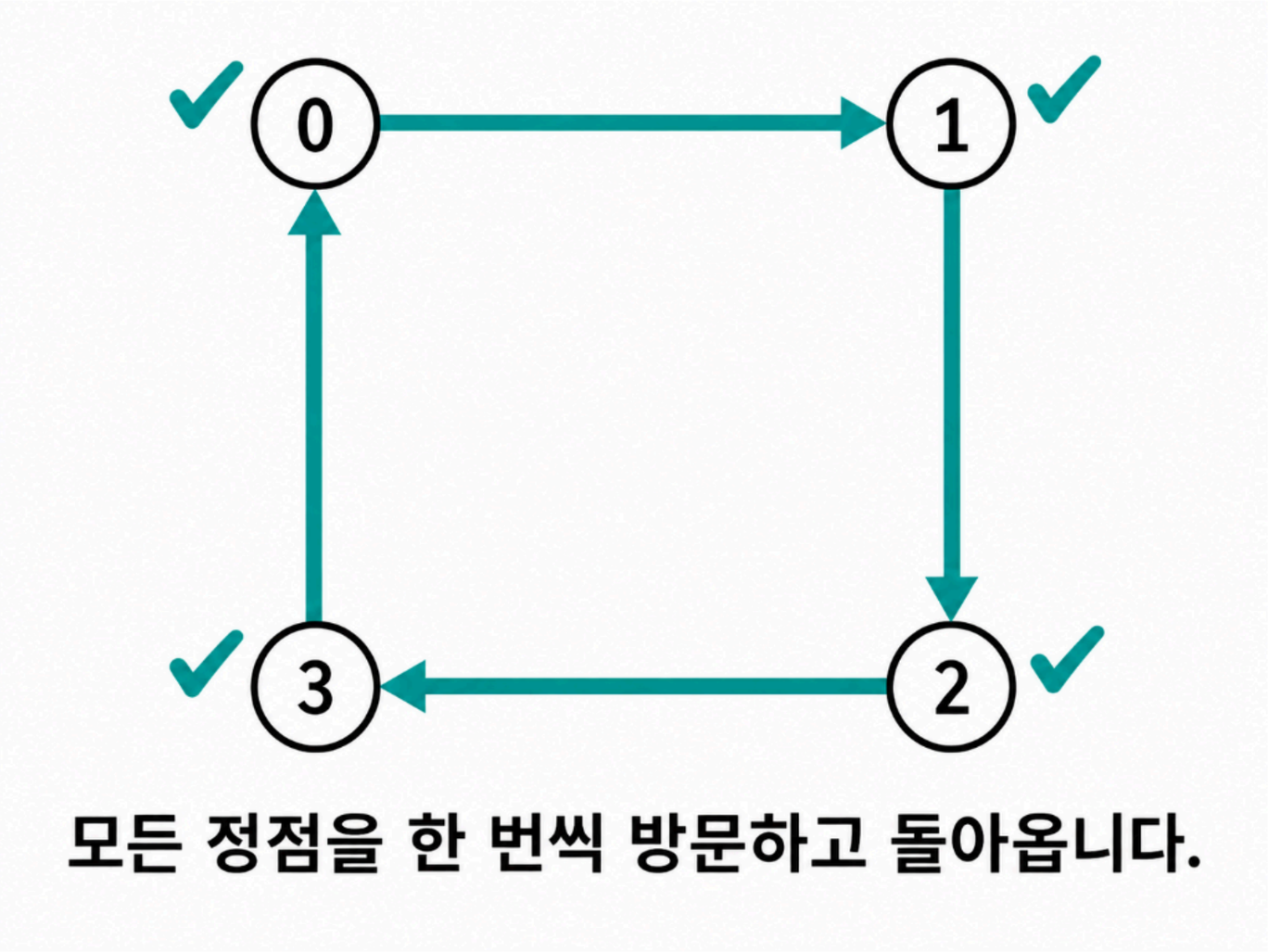
반환값

올바른 순회이면 정수 1, 아니면 정수 0이다.

선택·판정 조건

모든 번호는 0~n–1 범위여야 하고 중복되면 안 된다.
path에서 연속한 두 점 사이와 마지막·첫 점 사이에 모두 선이 있어야 한다.
시작점은 0이 아니어도 된다. path의 끝에는 시작점을 한 번 더 적지 않는다.

9-3 주어진 해밀턴 순회 확인



0→1→2→3을 방문한 뒤 3→0으로 돌아올 수 있어야 한다. 마지막 귀환 간선도 검사한다.

9-3 네 점을 한 바퀴 · 다른 점에서 출발

네 점을 한 바퀴

0–1–2–3–0인 사각형, path=[0,1,2,3]
반환값: 1
모든 점을 한 번씩 방문하고 3–0으로 돌아온다.

다른 점에서 출발

같은 사각형, path=[2,3,0,1]
반환값: 1
시작점이 2여도 모든 조건을 만족한다.

9-3 중복 번호 · 범위 밖 번호

중복 번호

같은 사각형, path=[0,1,1,3]
반환값: 0
1을 두 번 방문하고 2를 빠뜨렸다.

범위 밖 번호

같은 사각형, path=[0,1,2,9]
반환값: 0
점 번호는 0~3인데 9가 들어 있다.

9-3 귀환할 선이 없음

귀환할 선이 없음

0–1–2–3만 연결된 일자 그래프, path=[0,1,2,3]
반환값: 0
중간 이동은 가능하지만 마지막 3–0 선이 없다.

9-3 예제의 계산 과정

검사 순서	확인할 내용	통과 조건
1	점 번호	모두 0..n−1 안에 있음
2	방문 횟수	같은 점이 두 번 나오지 않음
3	중간 연결	0–1, 1–2, 2–3 존재
4	귀환 연결	3–0 존재

모든 점을 한 번씩 방문해 돌아오는 순회를 해밀턴 순회라고 부릅니다.

9-3 순서에 등장한 점과 이동한 선을 확인한다

먼저 path의 번호가 범위 안에 있는지와 중복 여부를 본다. 이 검사를 통과하면
점들은 빠짐없이 한 번씩 들어 있다. 이어서 실제 이동에 필요한 선들을 확인한다.
마지막 점에서 첫 점으로 돌아가는 선도 같은 방법으로 검사해야 한다.

9-3 코드에서 사용할 값

변수	뜻
seen[v]	점 v가 path에서 이미 나왔으면 1
path[i]	이번 점
path[(i+1)%n]	다음 점. 마지막에서는 첫 점
graph[u][v]	이 이동에 선이 있으면 1

경로를 새로 만드는 작업 없이, 제시된 경로를 그대로 검사합니다.

9-3 Python

```
def verify_hamiltonian(graph, path):
    n = len(graph)
    seen = [False]*n
    for v in path:
        if v < 0 or v >= n or seen[v]:
            return 0
        seen[v] = True
    for i in range(n):
        if not graph[path[i]][path[(i+1)%n]]:
            return 0
    return 1
```

행렬이 주어졌을 때 검사 시간 O(n), seen 배열 O(n)

9-3 C 함수의 인자

함수 원형

int verify_hamiltonian(const int graph[][8], int n, const int path[])

배열과 작업 공간

graph는 연결 표, n은 정점 수, path는 길이 n인 방문 순서이다.

9-3 C

```
int verify_hamiltonian(const int graph[][8], int n,
    const int path[]) {
    int seen[8]={0};
    for (int i=0; i<n; ++i) {
        int v=path[i];
        if (v<0 || v>=n || seen[v]) return 0;
        seen[v]=1;
    }
    for (int i=0; i<n; ++i)
        if (!graph[path[i]][path[(i+1)%n]]) return 0;
    return 1;
}
```

행렬이 주어졌을 때 검사 시간 O(n), seen 배열 O(n)

9-4 세 항짜리 논리 조건의 만족 가능성

변수 x_1부터 x_n에 각각 0(거짓) 또는 1(참)을 넣는다. 조건 하나는 정수 세 개로 표현한다. 양수 k는 x_k의 값이고, 음수 -k는 x_k의 값을 뒤집은 것이다. 한 조건의 세 값 중 하나라도 1이면 그 조건을 통과한다.

모든 조건을 동시에 통과하는 값 배정이 존재하면 1, 없으면 0을 반환하시오.

9-4 입력과 반환값

Python: satisfiable_3cnf(n, clauses)

n: 변수 수. 1~12의 정수
clauses: 길이 0~30의 조건 목록. 각 조건은 0이 아닌 정수 세 개이며 $1 \le |항| \le n$

반환값

가능한 값 배정이 하나라도 있으면 1, 전혀 없으면 0이다. 배정 목록을 반환할 필요는 없다.

선택·판정 조건

한 조건 안에서는 OR, 조건들 사이는 AND로 연결한다. 이런 형식을 3-CNF라고 한다.
예를 들어 [1,–2,3]은 x_1, 뒤집은 x_2, x_3 중 하나라도 참이면 통과한다.
같은 항이 한 조건에 여러 번 나올 수 있다. 조건 목록이 비었으면 1이다.

9-4 참으로 만들 수 있는 조건 · 서로 충돌하는 두 조건

참으로 만들 수 있는 조건

n=1, clauses=[[1,1,1]]
반환값: 1
x_1=1이면 세 항이 모두 1이어서 통과한다.

서로 충돌하는 두 조건

n=1, clauses=[[1,1,1],[-1,-1,-1]]
반환값: 0
첫 조건은 x_1=1, 둘째는 x_1=0을 요구하므로 동시에 통과할 수 없다.

9-4 음수 항의 역할 · 조건이 없음

음수 항의 역할

n=2, clauses=[[1,–2,1]]
반환값: 1
x_1=0, x_2=0이면 –2의 값은 1이므로 이 조건을 통과한다.

조건이 없음

n=2, clauses=[]
반환값: 1
위반할 조건이 없으므로 어떤 배정이나 가능하다.

9-4 서로 모순되는 두 조건

값 배정	조건 [1,1,1]	조건 [-1,-1,-1]	둘 다 통과?
$x_1=0$	실패	통과	아니요
$x_1=1$	통과	실패	아니요

위 두 조건을 동시에 만족하는 배정은 없으므로 이 예의 답은 0입니다.

9-4 변수 두 개면 네 배정을 검사한다

변수가 두 개이면 (x_1,x_2)는 (0,0),(1,0),(0,1),(1,1)의 네 경우이다. 배정 하나를 정할 때마다 모든 조건을 검사한다. 조건 하나라도 실패하면 다음 배정을 보고, 모두 통과한 배정이 나오면 1을 반환한다. n이 12 이하이므로 최대 4096개 배정을 확인한다.

0과 1 배정을 정수로 표현합니다

변수 두 개이면 가능한 배정은 네 개입니다.

mask	두 비트	x_2	x_1
0	00	0	0
1	01	0	1
2	10	1	0
3	11	1	1

오른쪽 끝이 x_1입니다. 0부터 3까지 보면 네 배정을 모두 봅니다.

코드의 비트 연산을 한 번 계산합니다

mask=5는 이진수 101, 즉 x_3=1, x_2=0, x_1=1입니다.

확인할 변수	오른쪽으로 이동	끝 비트만 읽기	값
x_1	101 >> 0 = 101	101 & 1	1
x_2	101 >> 1 = 10	10 & 1	0
x_3	101 >> 2 = 1	1 & 1	1

음수 항이면 읽은 값을 1-value로 뒤집습니다.

9-4 코드에서 사용할 값

입력 표현	뜻
양수 2	x_2의 값을 그대로 사용
음수 –2	x_2가 0이면 1, 1이면 0으로 뒤집음
[1,–2,3]	x_1, 반대 x_2, x_3 중 하나라도 참이면 통과
clauses	이런 세 값짜리 조건 묶음들의 목록

세 항을 OR로 묶은 조건들을 AND로 연결한 식을 3-CNF라고 부릅니다.

9-4 Python

```
def satisfiable_3cnf(n, clauses):
    for mask in range(1 << n):
        satisfied = True
        for clause in clauses:
            clause_ok = False
            for literal in clause:
                value = (mask >> (abs(literal)-1)) & 1
                if literal < 0: value = 1-value
                if value: clause_ok = True
            if not clause_ok:
                satisfied = False
                break
        if satisfied: return 1
    return 0
```

시간 $O(2^n \cdot \max(1,m))$, 배정 비트마스크 외 추가 공간 $O(1)$

9-4 C 함수의 인자

함수 원형

int satisfiable_3cnf(int n, const int clauses[][3], int m)

배열과 작업 공간

n은 변수 수, m은 조건 묶음 수이다. clauses의 각 행에는 항 3개를 넣는다.

9-4 C (1/2)

```
int satisfiable_3cnf(int n, const int clauses[][3], int m) {
    for (int mask=0; mask<(1<<n); ++mask) {
        int ok=1;
        for (int i=0; i<m; ++i) {
            int clause_ok=0;
            for (int j=0; j<3; ++j) {
                int lit=clauses[i][j], k=lit<0?-lit:lit;
                int value=(mask>>(k-1))&1;
                if (lit<0) value=1-value;
                if (value) clause_ok=1;
            }
            if (!clause_ok) {
                ok=0;
                break;
            }
```

9-4 C (2/2)

```
            }
            if (ok) return 1;
        }
        return 0;
}
```

시간 $O(2^n \cdot \max(1,m))$, 배정 비트마스크 외 추가 공간 $O(1)$

찾는 문제와 확인하는 문제

선택을 받았을 때

부분집합·정점 커버·순회의 검사 함수는 주어진 선택 하나만 확인한다.

선택을 찾아야 할 때

어떤 원소를 고를지, 어떤 순서로 방문할지를 바꾸어 보아야 한다. 3-CNF 함수는 값 배정을 직접 찾는다.

NP와 NP완전

NP는 예라는 답의 증거를 다항 시간에 확인할 수 있는 결정 문제의 집합이다. NP완전은 NP에 속하고 NP의 모든 문제를 다항 시간 변환으로 옮길 수 있는 문제이다.